

End-to-end Early Classification of Time Series in Non-Stationary Environments

Aurélien Renault

aurelien.renault@orange.com
Orange Research & AgroParisTech
Paris, France

Antoine Cornuéjols

antoine.cornuejols@agroparistech.fr
AgroParisTech UMR MIA-Paris
Palaiseau, France

Alexis Bondu

alexis.bondu@orange.com
Orange Research
Paris, France

Vincent Lemaire

vincent.lemaire@orange.com
Orange Research
Paris, France

Abstract

Early Classification of Time Series (ECTS) requires making accurate decisions as early as possible in inherently online and evolving environments. Yet, most existing methods assume stationarity and rely on separable designs, where classification and triggering are optimized independently—an assumption that fundamentally limits their adaptability under drift. In this work, we challenge this paradigm and study ECTS under non-stationary conditions. We provide the first systematic comparison between separable and end-to-end approaches across controlled drifting scenarios. Building on Reinforcement Learning, we introduce DQeND, a unified architecture that jointly learns representation, classification, and triggering decisions, while remaining directly comparable to state-of-the-art separable baselines. Across a wide range of drifts, DQeND demonstrates strong robustness across various non-stationary scenarios, consistently outperforming separable baselines. An ablation study further highlights that jointly updating representation and decision modules is critical to these gains. Overall, our results indicate that end-to-end learning can offer improved adaptation capabilities for ECTS in dynamic environments, and motivate further investigation of alternatives to separable designs.

CCS Concepts

• **Computing methodologies** → **Temporal reasoning**; *Online learning settings*; *Reinforcement learning*.

Keywords

Time Series, Online Learning, Reinforcement Learning

ACM Reference Format:

Aurélien Renault, Alexis Bondu, Antoine Cornuéjols, and Vincent Lemaire. 2026. End-to-end Early Classification of Time Series in Non-Stationary Environments. In . ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Motivation

Early Classification of Time Series (ECTS) has emerged as a critical paradigm for applications where timely decision-making is as important as predictive accuracy [2, 10, 21]. By allowing a model to issue a classification before observing the full time series, ECTS systems inherently operate under a trade-off between *earliness* and *accuracy*. While substantial progress has been made in designing accurate early classifiers, most existing approaches are developed and evaluated under the implicit assumption of a stationary data-generating process. However, this assumption rarely holds in practical deployments, where the process generating successive time series can be subject to distribution shifts and concept drifts. Despite the growing maturity of the ECTS literature, the impact of non-stationarity remains largely unexplored.

For instance, detecting toxic behavior on social media is subject to strong temporal constraints, as harmful content should be identified and filtered as early as possible. Moreover, the nature of such content can evolve with current events, e.g. during election periods, requiring early detection systems to continuously adapt to shifting definitions of what should be flagged.

One specificity of ECTS is that it can be viewed as a dual decision process: (i) the classification task, which aims to predict the class label based on partial observations, and (ii) the trigger (or stopping) task, which determines the optimal time at which a prediction should be emitted. The vast majority of existing methods adopt a separable design, where these two components are implemented independently and optimized one after another [10, 21]. Such a decomposition offers practical advantages, including flexibility, interpretability, and ease of optimization. However, it also implicitly assumes that classification and stopping decisions can be optimized independently.

This assumption becomes particularly limiting when the underlying time series distribution evolves over time. In such conditions, both the classification boundaries and the optimal stopping time may evolve in a coupled manner. A separable framework, by design, may struggle to capture this joint adaptation, as each component is typically optimized in a sequential fashion, e.g., encoder → classifier → trigger, preventing upstream modules from benefiting from feedback provided by downstream decisions. In contrast, end-to-end architectures alleviate these unidirectional optimization dependencies by enabling the learning signal, namely the incurred cost, to propagate throughout the entire decision pipeline. Yet, this

potential benefit of end-to-end learning for ECTS under distribution drift has yet to be investigated. Indeed, on one hand, existing work on end-to-end ECTS are limited in scope: they are often evaluated under stationary conditions and rarely benchmarked against strong separable baselines. On the other hand, existing work on non-stationarity remains in the separable scope and mainly focuses on adapting the triggering mechanism under changing decision costs, while assuming a fixed classification model [23]. Consequently, a question stands out: does jointly optimizing classification and stopping decisions improve robustness and adaptability when the underlying time series distribution itself evolves? And can we identify where the differences with separable methods, if any, manifest themselves?

In this work, we provide a systematic investigation of separable and end-to-end ECTS architectures under controlled non-stationarities. Our contributions are threefold:

- We conduct an extensive empirical comparison between separable and end-to-end paradigms, explicitly analyzing the regimes in which joint optimization becomes beneficial.
- Building on recent work on Reinforcement Learning (RL)-based ECTS [18, 22], we propose an end-to-end framework that jointly optimizes classification and triggering decisions, while being directly comparable to state-of-the-art separable baselines.
- We design controlled experimental protocols involving covariate shifts and concept drifts, demonstrating that end-to-end optimization improves robustness and adaptation capabilities across a wide range of evolving environments.

Together, our results provide new insights into the design of adaptive ECTS systems and highlight the importance of jointly learning prediction and stopping decisions in dynamic environments. Experiments of this paper are fully reproducible and the source code can be found at https://anonymous.4open.science/r/ects_concept_drift-8177

The remainder of this paper is organized as follows. Section 2 provides background on ECTS. Section 3 reviews related ECTS works. Section 4 describes the proposed RL-based end-to-end ECTS pipeline. Section 5 describes the experimental protocol, including the evaluation metrics, synthetic datasets, and the controlled cost drift scenarios. Finally, Section 6 presents and discusses the experimental results for several types of non-stationarities, and Section 7 concludes and discusses future work.

2 Background

2.1 Problem statement

In the ECTS problem, an input time series of size T is observed progressively. At time $t \leq T$, only a prefix $\mathbf{x}_t = \langle x_1, \dots, x_t \rangle$ is available, where $\{x_i\}_{(1 \leq i \leq t)}$ denotes the measurements (possibly multivariate). The full series $\mathbf{x}_T$ belongs to an unknown class $y \in \mathcal{Y}$. The task is to output a prediction $\hat{y} \in \mathcal{Y}$ for the class of the incoming time series at some time $\hat{t} \in [1, T]$ before the deadline T .

ECTS explicitly addresses a trade-off between predictive accuracy and earliness. The prediction correctness is measured by a misclassification cost $C_m(\hat{y}|y)$, and time pressure is modeled by a delay cost $C_d(t)$, which is assumed to be positive and typically

non-decreasing over time. Formally, we consider: $C_m : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$, $C_d : \mathbb{R}^+ \rightarrow \mathbb{R}$.

Let $s \in \mathcal{S}$ be some ECTS function belonging to a class of functions $\mathcal{S}$, whose output at time t when receiving $\mathbf{x}_t$ is:

$$s(\mathbf{x}_t) = \begin{cases} \emptyset & \text{if extra measures are queried;} \\ (\hat{y}_t, t) & \text{if prediction is triggered at } t < T; \end{cases} \quad (1)$$

Specifically, when the deadline is reached, i.e. $t = T$, the model is forced to return a prediction: $s(\mathbf{x}_T) = (\hat{y}_T, T)$.

In practice, ECTS is an online optimization problem: at each time t , the function $s(\mathbf{x}_t)$ must decide whether to predict or wait for more data, based only on the prefix $\mathbf{x}_t$. The s function induces a stopping time $\hat{t}$ on each incoming series that is the first time step for which s returns a non-null prediction. It can be defined as:

$$\hat{t} = \inf\{t \mid s(\mathbf{x}_t) \neq \emptyset\} \quad (2)$$

Once triggered, the remaining observations $\{\mathbf{x}_t\}_{(t < \hat{t} \leq T)}$ are not considered and prediction process is stopped. The goal is to choose $\hat{t}$ such that the incurred, time-dependent, loss $\mathcal{L}(s(\mathbf{x}_t), y) = C_m(\hat{y}_t|y) + C_d(\hat{t})$ is as small as possible.

From a machine learning perspective, the goal is to find a function $s \in \mathcal{S}$ that best optimizes the time-dependent loss function $\mathcal{L}$, minimizing the true risk over the data-generating process $\mathbb{P}(\mathcal{X} \times \mathcal{Y})$ [21]:

$$\arg \min_{s \in \mathcal{S}} \mathbb{E}_{(\mathbf{x}_T, y) \sim \mathbb{P}(\mathcal{X} \times \mathcal{Y})} [\mathcal{L}(s(\mathbf{x}_T), y)] \quad (3)$$

In order to solve the ECTS problem, a training set is made available that is composed of N labeled time series, denoted by $(\mathbf{x}_T^i, y^i)_{i \in [0, N]} \in (\mathcal{X} \times \mathcal{Y})$, where each series $\mathbf{x}_T = \langle x_1, \dots, x_T \rangle$ is complete and of the same size T , and associated with its label $y \in \mathcal{Y}$.

2.2 Online ECTS with drift

The standard ECTS formulation typically assumes that the data-generating process observed at training time remains unchanged at deployment. This implicitly entails that both the input distribution and the underlying predictive relationship are stationary. In practice, this assumption is often violated: the *actual* conditions encountered in production may differ from those seen during training and can evolve over time. Such variations may affect the input distribution (covariate shift), or the conditional relationship between inputs and labels (concept drift). Those may occur either within the duration of a single time series (i.e., along the time index $t \in [0, T]$) or across longer time scales, for instance due to seasonal or operational changes.

To explicitly account for these different temporal dimensions, we distinguish between the time index of a given time series and the deployment timeline. Let $u \in \mathbb{N}^+$ index successive incoming labeled time series $\{(\mathbf{x}_T, y)^u\}_{(1 \leq u \leq U)}$ observed during deployment, $(\mathbf{x}_T, y)^u$ is generated by some distribution $\mathcal{D}_u$ over $\mathcal{X} \times \mathcal{Y}$.

In the general case, the deployment process is characterized by several sources of uncertainty: (i) the distribution $\mathcal{D}_u$ is *unknown* to the learner at deployment; (ii) it can be *non-stationary*, i.e., drifting over deployment time u .

At deployment, an ECTS system repeatedly processes incoming time series drawn from the evolving sequence of distributions

$\{\mathcal{D}_u\}_{u \geq 1}$. The deployment performance of some ECTS function $s \in \mathcal{S}$ is then measured by:

$$R_U(s) = \sum_{u=1}^U \mathbb{E}_{(\mathbf{x}_T, y) \sim \mathcal{D}_u} [\mathcal{L}(s(\mathbf{x}_T), y)], \quad (4)$$

where the expectation is taken with respect to the current (and potentially drifting) data-generating process, as well as any additional sources of randomness.

The remainder of the paper instantiates and compares different controlled forms of non-stationarity, encompassing data and concept drifts and evaluates ECTS methods' robustness across this wide range of dynamic scenarios.

3 Related work

The ECTS literature can be broadly divided into *end-to-end* and *separable* approaches. End-to-end methods jointly learn a classifier and a triggering mechanism. While flexible, they are classifier-specific and often difficult to compare consistently across studies. In contrast, *separable* approaches decouple classification and decision triggering. This modular design dominates the literature and has been extensively evaluated [2, 10, 21].

3.1 Separable ECTS

Classification module. A common design choice in separable ECTS systems consists in discretizing the temporal axis and learning a collection of classifiers, each specialized for a given timestamp or subset of timestamps. This strategy enables the direct reuse of state-of-the-art Time Series Classification algorithms, while ensuring that predictions can be produced at different stages of the sequence. Among recent advances, the MiniROCKET pipeline [7] has emerged as a particularly strong choice, combining high predictive performance with excellent scalability. Its effectiveness stems from the use of a large number of fixed random convolutional kernels, coupled with simple yet informative aggregation functions, yielding rich representations at a very low computational cost. As such, MiniROCKET-based classifiers are frequently adopted as backbone models in modern separable ECTS pipelines [3, 22, 23].

Trigger module. On the decision side, a widely used baseline is the Proba Threshold strategy, which triggers a prediction as soon as the maximum posterior class probability exceeds a predefined threshold. Despite its simplicity, this approach has proven to be competitive across a variety of datasets and cost configurations, and is therefore often used as a reference method [21].

Beyond such myopic strategies, most high-performing trigger models fall into the class of *non-myopic* approaches, which explicitly account for the potential evolution of future observations when making decisions. The Economy family of methods [1, 4] follows this principle by triggering a decision when the expected cost at the current time step is minimal among all anticipated future costs. Similarly, Calimera [3] adopts an anticipation-based framework, leveraging regression models learned via backward induction to estimate future costs and guide the triggering decision.

Recently, RL has been introduced as a flexible alternative for modeling the triggering process. In particular, Alert [22] learns a triggering policy, based on a state space composed by handcrafted features,

obtained by probing the ensemble of calibrated MiniROCKET estimators. In addition to strong empirical performance, this approach is inherently well-suited to online adaptation and can naturally incorporate feedback from a deployment phase. For those reasons, the present work primarily builds upon this trigger model and investigates its integration within an end-to-end architecture, where the classifier is directly informed by the triggering signal.

3.2 End-to-end ECTS

Historically, the earliest works that can be considered end-to-end ECTS consist of a classifier-specific stopping criterion, directly integrated within a base Time Series Classification algorithm. A seminal example is the work of [28], which determines a unique minimum prediction length such that the predictions of a 1-nearest neighbor classifier remain the same as those obtained using a complete time series. In a similar vein, the EDSC algorithm [29] builds upon shapelet-based classification by associating each shapelet with a temporal penalty, thereby implicitly favoring patterns that occur earlier in the time series.

More recent advances in deep learning have enabled tighter integration between representation learning, classification, and decision timing. In particular, ELECTS [24] proposes an LSTM-based architecture, augmented by a stopping head that determines the probability of triggering. It is trained with a dedicated loss function that explicitly balances classification accuracy and earliness.

In parallel, the growing popularity of RL has led to a number of approaches that formulate ECTS as a sequential decision-making problem. Earlier work by [17, 18] introduced the use of Deep Q-Networks (DQN) for ECTS, modeling the triggering task as a value-based decision process. EARLIEST [11, 12] relies on a policy gradient method to learn a stopping policy jointly with a LSTM-based encoder and classifier. These RL-based methods offer a natural framework to capture the trade-off between earliness and accuracy, while enabling flexible adaptation to complex and potentially non-stationary environments.

4 DQeND: End-to-end RL for ECTS

In this section, the design choices for the proposed end-to-end ECTS pipeline, called DQeND, are described. In contrast to traditional separable approaches, where the different components are optimized independently, our architecture integrates them into a single decision process that can be trained and adapted holistically. The pipeline consists of two main modules: (i) an *encoder* that transforms raw time series prefixes into informative latent representations, and (ii) a RL-based *decision* module that simultaneously determines when to stop and which class to predict.

To enable fair comparisons with state-of-the-art separable methods and isolate the impact of end-to-end learning from architectural differences, we build our pipeline upon both ROCKET-type and Alert frameworks [6, 7, 22]. The global DQeND pipeline is illustrated in Figure 1.

4.1 Architecture

4.1.1 ELM Encoder. The design of the encoder module is guided by the objective of combining strong representational capacity with computational efficiency in an online setting. To this end, we

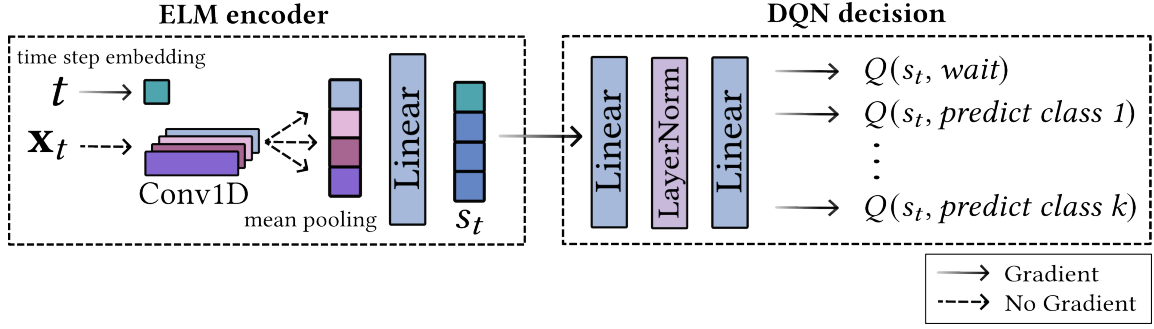


Figure 1: DQeND pipeline

draw inspiration from the principles underlying the widely adopted, ROCKET family of methods [6, 7], whose features are computed via a large number of random convolutions. More broadly, the idea of generating a large hidden representation of the input data using random weights originates from the Extreme Learning Machine (ELM) literature [13, 27].

At a high level, ELM-based approaches are single-layer neural networks. They rely on projecting input data into a high-dimensional feature space using a large set of randomly initialized and fixed parameters, and subsequently learning a lightweight mapping on top of this representation. Formally, the output of some ELM network with L hidden nodes can be defined as:

$$f_L(\mathbf{x}) = \sum_{i=1}^L \beta_i h_i(\mathbf{x}) \quad (5)$$

where $\beta = [\beta_1, \dots, \beta_L]$ is the learned weight matrix, and $\mathbf{h}(\mathbf{x}) = [h_1(\mathbf{x}), \dots, h_L(\mathbf{x})]$ the fixed random features for the i -th hidden node.

This paradigm has been shown to yield competitive performance across a wide range of time series classification tasks [15, 20], while maintaining a favorable computational profile due to the limited number of trainable parameters. Such properties are particularly desirable in online learning scenarios, where frequent updates must be performed under efficiency constraints.

Combining the ELM principles with the ROCKET-type of time series transformations, the proposed encoder implements a randomized feature extractor based on a single one-dimensional convolutional layer with frozen weights. Multiple kernel sizes k are employed in parallel in order to capture temporal patterns at different scales, thereby enriching the expressiveness of the resulting representation. The convolutional outputs are then aggregated through a mean pooling operation, producing a fixed-size embedding regardless of the input length. In particular, feature h_i can be obtained through the following operations:

$$h_i(\mathbf{x}_t) = \text{AvgPool}(\text{Conv1D}_k(\mathbf{x}_t)) \quad (6)$$

The convolutional filters are initialized by sampling their weights from a standard normal distribution and remain unchanged throughout training and deployment, ensuring a stable and inexpensive feature extraction process. To complement this representation, we additionally learn an explicit embedding of the time step, which

is concatenated to the ELM-based features. This allows the downstream decision modules to incorporate temporal position information alongside the extracted signal patterns, a key aspect for early classification tasks where the decision process is inherently time-dependent.

4.1.2 DQN Decision. The decision-making module is designed to remain directly comparable to strong separable baselines while enabling full end-to-end optimization. To this end, we adopt the same underlying architecture as the Alert framework [22], namely a Deep Q-Network (DQN)-based decision model [5, 26]. This choice ensures that any observed differences in performance can be attributed to the representation and training paradigm, rather than to architectural discrepancies in the decision process itself.

In contrast to separable approaches, Alert in particular, that typically rely on handcrafted features or classifier-derived confidence scores, the proposed module operates directly on the learned representation produced by the encoder. More precisely, the input state s_t is given by the embedding output at time t , allowing the decision module to leverage rich, task-adaptive features without additional engineering.

The action space is extended so as to jointly model the stopping decision and the class prediction. Specifically, we define $\mathcal{A} = \{\text{"wait"}, \text{"predict class 1"}, \dots, \text{"predict class K"}\}$, thereby unifying the triggering and classification tasks within a single decision process. At each time step, the selected action follows the standard greedy policy induced by the Q-function:

$$a_t = \arg \max_{a \in \mathcal{A}} Q(s_t, a) \quad (7)$$

The prediction mechanism is derived directly from the chosen action. If the model selects a prediction action, the corresponding class is immediately output. A specific handling is introduced for the terminal time step T : if the selected action remains "wait", the model is forced to emit a prediction, corresponding to the class associated with the highest Q-value among the non-waiting actions. Formally, the predicted label is defined as:

$$\hat{y}_t(a_t) = \begin{cases} \emptyset & \text{if } a_t = \text{"wait"} \text{ and } t < T \\ \arg \max_{a \in \mathcal{A} \setminus \{\text{"wait"}\}} Q(s_t, a) & \text{if } a_t = \text{"wait"} \text{ and } t = T \\ a_t & \text{if } a_t \neq \text{"wait"} \end{cases} \quad (8)$$

4.2 Training (offline)

For stability reasons, learning is performed offline in a batch setting, where the model is trained on a fixed collection of fully labeled time series without requiring active exploration during training. In particular, the overall training procedure is kept close to that of Alert [22].

The RL objective is defined through a reward function that captures the trade-off between earliness and accuracy. At each time step, the reward is expressed as the temporal variation of the delay cost when the agent decides to wait, and additionally incorporates the misclassification cost when a prediction is made. Formally, the reward is defined as:

$$r(t) = \begin{cases} -\Delta C_d(t) & \text{if } a_t = \text{"wait"} \\ -C_m(\hat{y}_t | y) - \Delta C_d(t) & \text{if } a_t \neq \text{"wait"} \end{cases} \quad (9)$$

where $-\Delta C_d(t) = C_d(t) - C_d(t-1)$, with $C_d(0) = 0$.

The gradient of the DQN objective, corresponding to the Bellman error, is propagated through the entire DQeND pipeline, updating all parameters associated with the shaded arrows in Figure 1. Finally, the model selection procedure is kept identical to that of [22].

4.3 Deployment (online)

The proposed framework naturally extends to an online adaptation setting, where both the encoder and the decision modules are jointly updated based on the feedback induced by the decisions. In contrast to separable approaches, this joint update enables the representation itself to evolve in response to non-stationary conditions, allowing for a tighter coupling between feature extraction and decision-making.

At deployment time, each observed interaction is stored in a replay buffer. The pipeline is then updated by sampling mini-batches of transitions from this buffer, following standard DQN optimization procedures.

When moving from offline training to online adaptation, exploration is explicitly reintroduced in order to account for potential distribution shifts. Concretely, we adopt an ϵ -greedy strategy, where with probability ϵ a random action is selected, and with probability $1 - \epsilon$ the greedy action, i.e. the one with maximum Q -value, is chosen. In the exploration phase, actions are sampled uniformly from the action space, ensuring equal probability across all possible decisions (e.g., 1/3 for each action in the binary classification case). In all experiments, the exploration rate is fixed to $\epsilon = 5\%$. Details on all the hyperparameters' values can be found in Table 1.

5 Experimental Protocol

This section describes the experimental protocol used to evaluate the proposed online ECTS methods under non-stationarities. We detail the evaluation metrics, the data generation process, the drifts scenarios, and the evaluated methods. All these elements are fixed prior to the analysis of the results presented in Section 6.

5.1 Evaluation Metrics

All online methods are evaluated using an *eval-then-update* (prequential) protocol [8]. At each deployment step u , a time series and its associated deployment-time costs are first used to evaluate the

Table 1: DQeND hyperparameters

Encoder	kernel sizes	{3, 6, 9, 12}
	nb. of kernels	{128, 64, 32, 16}
	embedding dim.	64
	time embedding dim.	2
Decision	nb. hidden layers	1
	hidden dim.	64
	activation function	tanh
Training	learning rate	1e-3
	nb. epochs	100
	batch size	256
Deployment	learning rate	1e-4
	exploration rate ϵ	0.05
	batch size	64

current model. The observed loss is then used to update the models according to its update strategy.

To study the impact of different trade-offs between misclassification and delay costs, we use the standard weighted formulation of the average cost, along with the cost functions definitions used by [1, 3, 21]:

$$\begin{aligned} AvgCost_\alpha &= \frac{1}{N} \sum_{i=1}^N C_m(\hat{y}_i | y_i) + C_d(\hat{t}_i) \\ &= \frac{1}{N} \sum_{i=1}^N \alpha \mathbb{1}(\hat{y} \neq y) + (1 - \alpha) \frac{t}{T} \end{aligned}$$

where $\alpha \in [0, 1]$ controls the relative importance of accuracy and earliness. Throughout this work, as in [16, 19], we fix $\alpha = 0.8$.

The evaluation relies on a set of three metrics derived from the standard *AvgCost*, in order to capture complementary aspects of performance under non-stationary conditions:

- First, we report the **cumulative cost**, defined as the total sum of incurred costs over all deployment examples.
- Second, we consider the **hold-out AvgCost drift**, computed on a disjoint evaluation set that is subject to the same drift conditions as the current deployment step. Unlike the raw cumulative deployment cost, which may exhibit high variance due to transient effects or small batches, this metric provides a more stable estimate of performance under the current time series distribution.
- Finally, we report the **hold-out AvgCost original**, evaluated on a separate hold-out set drawn from the original (training-time) distribution. This complementary metric assesses the extent to which a method preserves performance on the base concept, thereby offering insight into its ability to balance adaptation and retention; often referred to as the stability-plasticity trade-off [14, 25].

5.2 Data Generation

Studying non-stationarity in ECTS requires benchmarks where the difficulty of the task and the nature of the drift can be precisely controlled. In particular, the ability to independently manipulate factors such as noise intensity, temporal location of discriminative patterns, or class structure is essential. The MNIST-1D [9] dataset

naturally fulfills these requirements through its synthetic generation process, making it particularly well-suited for the controlled drift scenarios considered in this work. The MNIST-1D dataset is a ten-class classification problem, which has been binarized in this work, based on univariate time series of fixed length $T = 100$. Each generated time series contains a class-specific pattern representing the shape of a digit, embedded at a random position. Training phase is conducted using 1,000 examples (700 for training and the 300 left to compute the hold-out *AvgCost* original); at each deployment step u , 1,500 new (drifted) examples are generated, among which 500 are used for updates and 1,000 to compute the hold-out *AvgCost* drift.

5.3 Drift Scenarios

To evaluate the robustness of online ECTS methods to non-stationarities, we consider controlled variations of input data and concept. Non-stationary patterns occur across deployment steps u , while the time index $t \in [0, T]$ remains associated with the progression within an individual time series.

We first distinguish two temporal patterns of variation:

- **Incremental (I) drifts** occur when the drifted variable(s) evolve smoothly during deployment.
- **Abrupt (A) drifts** occur when the drifted variable(s) suddenly switch from the training regime to a new deployment regime.

We consider three variables over which to intervene in order to generate drifts:

- **Template Location (*loc*)** defines the location of the true class pattern within a time series.
- **Gaussian Noise (*noise*)** defines the scale of the Gaussian noise to be applied over a time series.
- **Class shuffling (*class*)** defines the set of sub-classes characterizing the two binary (meta-)classes.

Combining these two dimensions yields four experimental scenarios:

- (1) ***I_loc***: the class patterns are first located at the very beginning of the time series; it progressively shifts so that it reaches the end of the series at the end of deployment.

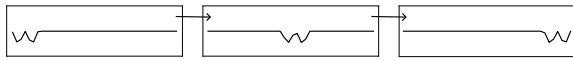


Figure 2: The *I_loc* scenario.

- (2) ***I_noise***: the scale of the *iid* Gaussian noise becomes progressively greater, following a log spaced schedule from 0.01 to 1.

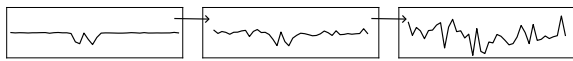


Figure 3: The *I_noise* scenario.

- (3) ***A_loc***: the class patterns are first located within the first quarter of the series; after five deployment steps, they are shifted to the last quarter.



Figure 4: The *A_loc* scenario.

- (4) ***A_class***: the set of sub-classes defining the two main classes is completely resampled, with a 5% probability of occurring at each deployment step.

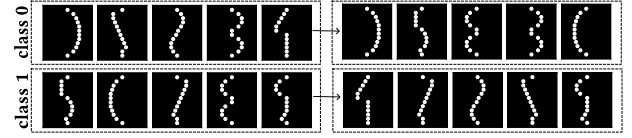


Figure 5: The *A_class* scenario.

5.4 Evaluated Methods

Separable baselines. As highlighted in Section 4, Alert constitutes the comparison reference for the proposed end-to-end method. Thus, all the separable models described in this section are based on: (i) a *classifier* that shares the same architecture as the ELM encoder described in Section 4.1.1, followed by a linear classification layer and (ii) a *trigger* which is Alert, as described in the original paper [22], including the handcrafted input space. In what follows, several ways of adapting separable baselines to online environments, depending on the addressed drift type, are presented.

• Common baselines:

- bronze*: both the classifier and the trigger model remain frozen during deployment.
- silver*: both the classifier and the trigger model are being trained on the concept on which they will be evaluated. Act as an upper bound, not realistic in practice.

• Incremental drifts:

- online/freeze*: the classifier is updated online, after each deployment step, the trigger model remains frozen.
- online/online*: both the classifier and the trigger model are updated online, after each deployment step. The data batch is split in two so that each module is updated based on one half of the batch.

• Abrupt drifts:

- retrain/freeze*: the classifier is retrained from scratch, at a given frequency, the trigger model remains frozen.
- retrain/retrain*: both the classifier and the trigger model are retrained from scratch, at a given frequency. The data batch is split in two so that each module is trained on the basis of one half of the batch.

End-to-end. The following end-to-end methods have been adapted so that they may be updated online: ELECTS [24] and EARLIEST [11].

5.5 Implementation details

The deployment phase consists of $U = 100$ steps of 500 examples each. For the *retrain* baselines, the retraining frequency has been set to 20. For both ELECTS and EARLIEST algorithms, training phase is conducted over 500 epochs, the deployment batch size has been set to 64 and the learning rate to $5e-3$. All experiments have been run on CPU, with up to five threads to perform cross-validation, when needed. Results that follow consist of the mean taken over five different random seeds. For all other details, we make the code available at: https://anonymous.4open.science/r/ects_concept_drift-8177

6 Results

In this section, we analyze the ECTS online performances under the two types of drifts presented in Section 5.

6.1 Incremental drifts

Incremental Location (I_{loc}). This first experiment investigates a gradual shift of discriminative patterns toward later positions in the deployment time series.

As shown in Figure 6a, end-to-end approaches clearly outperform separable baselines, consistently achieving the lowest cumulative costs (top panel). In particular, DQeND ranks first overall and, together with ELECTS, best preserves performance on the original concept (bottom panel), demonstrating a strong balance between adaptation and retention. In addition, the qualitative analysis in Figure 7 (top panel) shows that end-to-end models effectively adapt their triggering policies by progressively delaying predictions, illustrated by the increasing light-blue earliness curve of each box, as the discriminative patterns shift later in the series.

In contrast, both Alert-based separable methods degrade significantly as informative patterns move later in the sequences (middle panel, right), highlighting the limitations of decoupled classification and triggering under temporal drift. Freezing the Alert trigger further leads to unstable behavior throughout deployment, whereas updating both classifier and trigger slightly improves stability but still fails to improve global performance.

Incremental Noise (I_{noise}). This second experiment evaluates robustness under a gradual increase of additive Gaussian noise, progressively degrading the signal-to-noise ratio and the reliability of discriminative patterns.

As displayed in Figure 6b, end-to-end approaches again achieve the best cumulative costs, with DQeND ranking first, closely followed by ELECTS. Despite similar overall performance, analysing Figure 7, bottom panel, indicates that the two methods exhibit distinct adaptation behaviors. ELECTS follows a conservative strategy, initially delaying predictions to preserve accuracy while the signal remains informative, before progressively shifting toward earlier decisions as noise becomes dominant. In contrast, DQeND rapidly converges toward an early-prediction regime once accuracy approaches random performance levels (explaining the plateau shape in Figure 6b, middle panel), effectively minimizing delay costs when waiting no longer provides useful information. This difference in the adaptation strategy is illustrated by a vertical shift of the earliness curve of both algorithms.

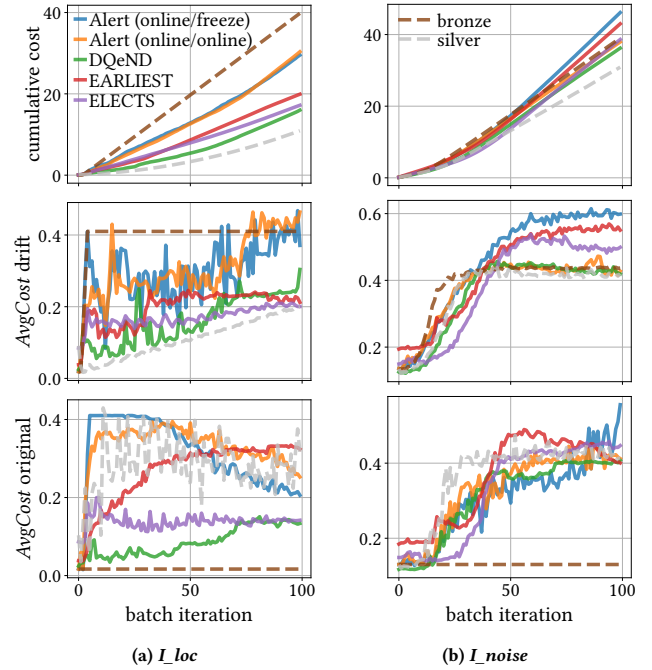


Figure 6: Incremental drifts, deployment’s metrics, averaged over five seeds. From top to bottom: the raw cumulative cost; the hold-out AvgCost, obtained on unseen drifted data; the hold-out AvgCost original, obtained using the fixed test set from training phase. UPPERCASE approaches refers to end-to-end methods while lowercase indicates separable ones.

Regarding separable approaches, the same trends as in the incremental location drift experiment can be observed. In particular, the frozen Alert trigger again produces unstable behavior, whereas updating both classifier and trigger significantly improves robustness. The *online/online* variant achieves performance close to DQeND under low to moderate noise levels (Figure 6b, middle panel, left), although the end-to-end model remains slightly more robust, suggesting that unified architectures better exploit weak residual signal under mild perturbations.

6.2 Abrupt drifts

Abrupt Location (A_{loc}). This experiment evaluates an abrupt drift scenario in which the discriminative pattern is suddenly shifted from the beginning to the end of the time series.

As shown in Figure 8a, top-middle panel, the RL-based approaches adapt most effectively to this sudden change. Both DQeND and EARLIEST rapidly shift from early to late predictions, successfully adjusting their triggering policies to the new pattern location. RL-based methods also best preserve performance on the original concept (Figure 8a, bottom), illustrating their ability to balance adaptation and retention. In contrast, ELECTS struggles to substantially delay its decisions at deployment, with trigger points rarely extending beyond half of the sequence. This limited shift highlights a lack

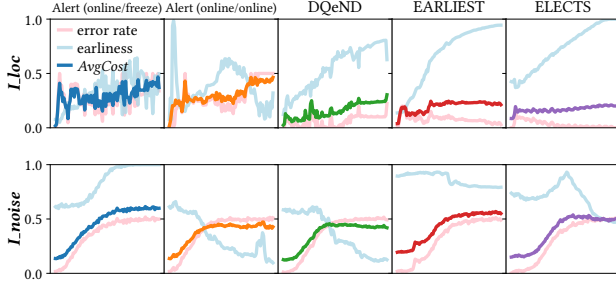


Figure 7: Hold-out AvgCost drift: detailed performances. The figures display decomposition of the hold-out AvgCost drift into both earliness, i.e. the average proportion of the time series used, and error rate, during the deployment phase.

of plasticity, as the model remains biased toward its initial regime and fails to fully adapt to the new optimal decision timing.

For the Alert-based separable baselines, the importance of online adaptation becomes even more evident. Freezing the trigger model can lead to catastrophic failures, as the decision policy becomes completely misaligned with the shifted distribution. Updating the trigger substantially improves stability and eventually recovers appropriate triggering behavior, although adaptation remains slower than with end-to-end approaches.

Sub-classes shuffling (A_{class}). This final experiment considers a highly non-stationary setting where the composition of meta-classes is randomly reshuffled at irregular intervals, directly altering the underlying class structure rather than the time series distribution.

In Figure 8b, ELECTS achieves the best overall performance, consistently recovering after each drift event. The RL-based approaches, DQeND and EARLIEST, closely follow and remain competitive. Noticeably, RL-based approaches suffer from a stronger constraint: unlike ELECTS and separable baselines, they only observe the data up to the estimated trigger time rather than the full time series. This makes adaptation to abrupt-type drifts substantially more challenging, a strong exploration rate being thus critical to efficiently recover. A more detailed analysis via Figure 9, bottom panel, nevertheless reveals a limitation of DQeND in this regime. Under repeated abrupt drifts, its triggering policy progressively collapses toward systematically early predictions, represented by the light-blue earliness curve, which keeps decreasing. This highlights the instability of the fully RL-based pipeline, which likely results from the accumulation of policy updates across successive distribution shifts. In contrast, ELECTS and EARLIEST display more stable earliness profiles (almost flat earliness curve) throughout deployment, reflecting a more conservative adaptation strategy.

Within this setting, separable baselines achieve performance comparable to the RL-based approaches, although being retrained at fixed, predetermined intervals. Interestingly, updating the trigger model brings only limited improvements over keeping it frozen.

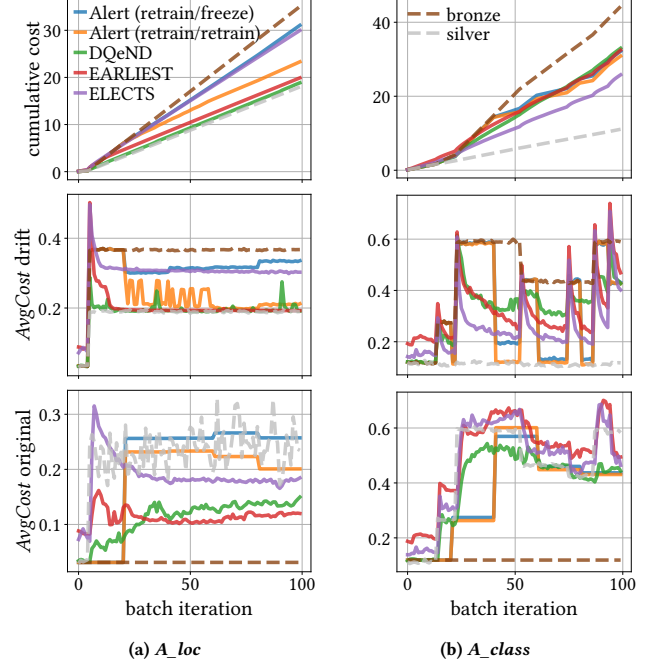


Figure 8: Abrupt drifts, deployment's metrics, averaged over five seeds. From top to bottom: the raw cumulative cost; the hold-out AvgCost, obtained on unseen drifted data; the hold-out AvgCost original, obtained using the fixed test set from training phase. UPPERCASE approaches refers to end-to-end methods while lowercase indicates separable ones.

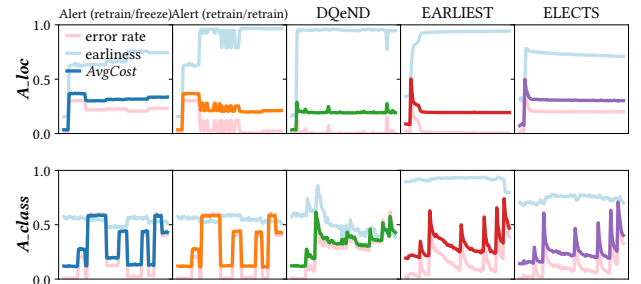


Figure 9: Hold-out AvgCost drift: detailed performances. The figures display decomposition of the hold-out AvgCost drift into both earliness, i.e. the average proportion of the time series used, and error rate, during the deployment phase.

6.3 Ablation studies

To further assess the benefits of the proposed end-to-end design, we conduct an ablation study in which the encoder and decision modules of DQeND are alternatively frozen during adaptation. This allows isolating the respective contributions of representation learning and decision-making in the presented non-stationary settings.

The results, reported in Table 2, show that jointly updating both modules consistently yields the best performance across most scenarios. In contrast, partially frozen variants lead to noticeable degradations, confirming the importance of coordinated adaptation. Among the two, freezing the decision module results in the most significant performance drop. This observation is further supported by the fact that the decision-making module accounts for most of the learnable parameters in the proposed architecture.

Interestingly, freezing the encoder has a more limited impact in the abrupt drift settings considered in our experiments. This suggests that a well-trained representation can remain sufficiently informative, provided that the decision module is allowed to adapt. Such behavior is intuitive in the *A_class* scenario, where the input distribution itself remains unchanged, despite shifts in class composition. In the *A_loc* setting, although the temporal position of the discriminative pattern changes, the nature and scale of the signal remain consistent, and the drift range remains limited, enabling the decision module to adjust its policy without requiring an update of the underlying representation.

Table 2: (a): The mean percentage difference between the cumulative cost of full updated DQeND and the partially frozen version, over five random seeds. (b): The mean of the median percentage difference of the hold-out AvgCost drift of the full updated DQeND and partially frozen version, over five random seeds. Negative figures indicate degraded performance compared to original DQeND.

	freeze encoder		freeze decision	
	(a)	(b)	(a)	(b)
<i>I_loc</i>	-32.42%	-36.44%	-36.61%	-31.87%
<i>I_noise</i>	-11.29%	-4.04%	-19.28%	-6.63%
<i>A_loc</i>	1.38%	-0.34%	-43.52%	-45.30%
<i>A_class</i>	1.32%	2.14%	-18.18%	-4.39%

7 Conclusion

In this paper, we investigated the relevance of end-to-end learning for Early Classification of Time Series under evolving time series distributions. To address this question, we conducted extensive experiments within a controlled synthetic drift framework, systematically comparing end-to-end approaches against the widely used separable paradigm. To this end, we proposed DQeND, a Reinforcement Learning-based end-to-end ECTS architecture designed to remain closely comparable to state-of-the-art separable baselines. Across a broad range of gradual and abrupt non-stationarities, end-to-end approaches consistently demonstrated stronger robustness and adaptation capabilities than separable competitors, whose behavior often remained unstable and limited under drift. In particular, DQeND achieved strong online performance while preserving adapted behaviors on the original training concept, highlighting its ability to balance adaptation and retention in dynamic environments. More broadly, our results emphasize the benefits of jointly optimizing representation learning and triggering decisions in the context of ECTS.

Future work could include a more rigorous theoretical analysis of the stability-plasticity trade-off in ECTS, as well as the study of

additional forms of non-stationarity, such as drifting cost functions or evolving class priors. Finally, beyond the controlled experimental setting considered in this work, the proposed framework provides a practical foundation for deployment in more complex real-world streaming environments.

A Variability report

In this section, the evaluation metrics displayed in Section 6 are detailed in Tables 3, 4, 5 and variability is reported.

Table 3: The mean cumulative cost and standard deviation, over five random seeds. Bold value indicates the best average performing method.

	<i>I_loc</i>	<i>I_noise</i>	<i>A_loc</i>	<i>A_class</i>
Alert(online/freeze)	29.02 ± 4.08	46.14 ± 0.3	31.06 ± 11	32.15 ± 1.27
Alert(online/online)	31.87 ± 5.49	37.92 ± 0.83	23.32 ± 1.88	30.86 ± 0.2
DQeND	16.18 ± 3.40	36.2 ± 1.04	18.86 ± 0.55	32.98 ± 3.88
EARLIEST	20.53 ± 6.54	43 ± 3.37	19.90 ± 0.79	32.44 ± 5.88
ELECTS	17.33 ± 1.81	38.63 ± 1.98	29.98 ± 12.06	25.79 ± 3.52

Table 4: The mean hold-out AvgCost drift and standard deviation, over all the deployment steps and five random seeds. Bold value indicates the best average performing method.

	<i>I_loc</i>	<i>I_noise</i>	<i>A_loc</i>	<i>A_class</i>
Alert(online/freeze)	0.3 ± 0.04	0.46 ± 0.003	0.31 ± 0.11	0.32 ± 0.01
Alert(online/online)	0.31 ± 0.07	0.38 ± 0.009	0.23 ± 0.02	0.31 ± 0.002
DQeND	0.16 ± 0.03	0.36 ± 0.01	0.19 ± 0.01	0.33 ± 0.04
EARLIEST	0.2 ± 0.06	0.43 ± 0.03	0.2 ± 0.01	0.32 ± 0.06
ELECTS	0.17 ± 0.02	0.39 ± 0.02	0.3 ± 0.12	0.26 ± 0.04

Table 5: The mean hold-out AvgCost original and standard deviation, over all the deployment steps and five random seeds. Bold value indicates the best average performing method.

	<i>I_loc</i>	<i>I_noise</i>	<i>A_loc</i>	<i>A_class</i>
Alert(online/freeze)	0.31 ± 0.02	0.31 ± 0.04	0.21 ± 0.04	0.37 ± 0.03
Alert(online/online)	0.33 ± 0.06	0.33 ± 0.02	0.18 ± 0.02	0.37 ± 0.003
DQeND	0.09 ± 0.04	0.31 ± 0.01	0.11 ± 0.02	0.4 ± 0.04
EARLIEST	0.27 ± 0.07	0.36 ± 0.01	0.11 ± 0.07	0.51 ± 0.01
ELECTS	0.14 ± 0.07	0.33 ± 0.01	0.19 ± 0.1	0.48 ± 0.02

B Runtime analysis

Computation times are reported in Table 6. Overall, ELECTS and EARLIEST are the fastest approaches, both during training and deployment. Unlike Alert and DQeND, which rely on convolution-based encoders, these methods are LSTM-based and therefore benefit from the CPU-only experimental setup used in this work. In practice, convolutional approaches would likely reduce this gap when leveraging GPU acceleration. For Alert-based methods indeed, most of the training time is spent fitting the ELM-based classifier rather than the trigger module itself, confirming the relatively low computational overhead of the RL trigger. In contrast, DQeND exhibits the highest training and update times.

More broadly, the deployment times of DQeND remain significantly longer. This is consistent with the known sample inefficiency of value-based RL algorithms, whose repeated updates and replay-based optimization can constitute computational limitations in online settings.

Table 6: Computation times (in s.). The deployment are displayed on a *per batch* basis. The total time accounts for the all 100 deployment steps. The fit time of the Alert-based methods are decomposed over (*classifier fit time*, *trigger fit time*).

	Training Fit	Deployment (<i>per batch</i>)		Total ↓
		Inference	Update	
ELECTS	31.89	0.026	0.05	39.22
EARLIEST	119.25	0.23	0.09	138.98
Alert (<i>online/online</i>)	(78.02, 46.55)	2.39	2.45	612.75
Alert (<i>retrain/retrain</i>)	(78.02, 46.55)	2.71	3.59	759.86
DQeND	248.74	2.90	5.75	1124.31

References

- [1] Youssef Achenchabe, Alexis Bondu, Antoine Cornuéjols, and Asma Dachraoui. 2021. Early classification of time series: Cost-based optimization criterion and algorithms. *Machine Learning* 110, 6 (2021), 1481–1504.
- [2] Charilaos Akasiadis, Evgenios Kladis, Petro-Foti Kamberi, Evangelos Michelioudakis, Elias Alevizos, and Alexander Artikis. 2024. A Framework to Evaluate Early Time-Series Classification Algorithms.. In *EDBT*. 623–635.
- [3] Jakub Michal Bilski and Agnieszka Jastrzebska. 2023. CALIMERA: A new early time series classification method. *Information Processing & Management* 60, 5 (2023), 103465.
- [4] Asma Dachraoui, Alexis Bondu, and Antoine Cornuéjols. 2015. Early classification of time series as a non myopic sequential decision making problem. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2015, Porto, Portugal, September 7–11, 2015, Proceedings, Part I* 15. Springer, 433–447.
- [5] Peter Dayan and CJCH Watkins. 1992. Q-learning. *Machine learning* 8, 3 (1992), 279–292.
- [6] Angus Dempster, François Petitjean, and Geoffrey I Webb. 2019. ROCKET: exceptionally fast and accurate time series classification using random convolutional kernels. *arXiv preprint arXiv:1910.13051* (2019).
- [7] Angus Dempster, Daniel F Schmidt, and Geoffrey I Webb. 2021. Minirocket: A very fast (almost) deterministic transform for time series classification. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 248–257.
- [8] João Gama, Raquel Sebastião, and Pedro Pereira Rodrigues. 2009. Issues in evaluation of stream learning algorithms. In *SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '09)*. Association for Computing Machinery, 329–338.
- [9] Sam Greydanus and Dmitry Kobak. 2020. Scaling down deep learning with mnist-1d. *arXiv preprint arXiv:2011.14439* (2020).
- [10] Ashish Gupta, Hari Prabhat Gupta, Bhaskar Biswas, and Tanima Dutta. 2020. Approaches and applications of early classification of time series: A review. *IEEE Transactions on Artificial Intelligence* 1, 1 (2020), 47–61.
- [11] Thomas Hartvigsen, Cansu Sen, Xiangnan Kong, and Elke Rundensteiner. 2019. Adaptive-halting policy network for early classification. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 101–110.
- [12] Thomas Hartvigsen, Cansu Sen, Xiangnan Kong, and Elke Rundensteiner. 2020. Recurrent halting chain for early multi-label classification. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1382–1392.
- [13] Guang-Bin Huang, Qin-Yu Zhu, and Chee-Kheong Siew. 2006. Extreme learning machine: theory and applications. *Neurocomputing* 70, 1–3 (2006), 489–501.
- [14] Dongwan Kim and Bohyung Han. 2023. On the stability-plasticity dilemma of class-incremental learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 20196–20204.
- [15] Huaping Liu, Lianzhi Yu, Wen Wang, and Fuchun Sun. 2016. Extreme learning machine for time sequence classification. *Neurocomputing* 174 (2016), 322–330.
- [16] Junwei Lv, Yuqi Chu, Jun Hu, Peipei Li, and Xuegang Hu. 2023. Second-order Confidence Network for Early Classification of Time Series. *ACM Transactions on Intelligent Systems and Technology* (2023).
- [17] Coralie Martinez, Guillaume Perrin, Emmanuel Ramasso, and Michèle Rombaut. 2018. A deep reinforcement learning approach for early classification of time series. In *2018 26th European Signal Processing Conference (EUSIPCO)*. IEEE, 2030–2034.
- [18] Coralie Martinez, Emmanuel Ramasso, Guillaume Perrin, and Michèle Rombaut. 2020. Adaptive early classification of temporal sequences using deep reinforcement learning. *Knowledge-Based Systems* 190 (2020), 105290.
- [19] Usue Mori, Alexander Mendiburu, Eamonn Keogh, and Jose A Lozano. 2017. Reliable early classification of time series based on discriminating the classes over time. *Data mining and knowledge discovery* 31 (2017), 233–263.
- [20] Jin-Man Park and Jong-Hwan Kim. 2017. Online recurrent extreme learning machine and its application to time-series prediction. In *2017 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1983–1990.
- [21] Aurélien Renault, Alexis Bondu, Antoine Cornuéjols, and Vincent Lemaire. [n. d.]. Early Classification of Time Series: A Survey and Benchmark. *Transactions on Machine Learning Research* ([n. d.]).
- [22] Aurélien Renault, Alexis Bondu, Antoine Cornuéjols, and Vincent Lemaire. 2025. Deep Reinforcement Learning based Triggering Function for Early Classifiers of Time Series. *arXiv preprint arXiv:2502.06584* (2025).
- [23] Aurélien Renault, Alexis Bondu, Antoine Cornuéjols, and Vincent Lemaire. 2026. Early Classification of Time Series in Non-Stationary Cost Regimes. *arXiv preprint arXiv:2602.00918* (2026).
- [24] Marc Rußwurm, Nicolas Courty, Rémi Emonet, Sébastien Lefèvre, Devis Tuia, and Romain Tavenard. 2023. End-to-end learned early classification of time series for in-season crop type mapping. *ISPRS Journal of Photogrammetry and Remote Sensing* 196 (2023), 445–456.
- [25] Miquel Serra-Perello and Alberto Ortiz. 2025. Incremental Learning Methodologies for Addressing Catastrophic Forgetting: Analysis and Experimental Evaluation. *Journal of Artificial Intelligence Research* 83 (2025).
- [26] Richard S Sutton and Andrew G Barto. 2018. *Reinforcement learning: An introduction*. MIT press.
- [27] Jian Wang, Siyuan Lu, Shui-Hua Wang, and Yu-Dong Zhang. 2022. A review on extreme learning machine. *Multimedia Tools and Applications* 81, 29 (2022), 41611–41660.
- [28] Zhengzheng Xing, Jian Pei, and S Yu Philip. 2009. Early Prediction on Time Series: A Nearest Neighbor Approach.. In *IJCAI*. Citeseer, 1297–1302.
- [29] Zhengzheng Xing, Jian Pei, Philip S Yu, and Ke Wang. 2011. Extracting interpretable features for early classification on time series. In *Proceedings of the 2011 SIAM international conference on data mining*. SIAM, 247–258.